

Project 4 Report

CS 513 Computer Networks, Craig E. Wills

Due: December 13, 2025

Alexander Kraemling, Dylan Vo

Names:

Alexander Kraemling, Dylan Vo

Selected project:

We selected the standard Link-State Routing and Distance-Vector Routing Algorithm project. For this project, Alexander mainly worked on parsing user input\ building and editing the graph, as well as the Distance Vector Routing Algorithm, and Dylan mainly worked on the Link State Routing Algorithm.

Link State Implementation

The Link-State Routing Algorithm uses Dijkstra's Algorithm to compute the shortest-path tree once the router has a complete map of the network. Dijkstra provides the minimum cost and predecessor for each destination, and then the router derives the next hop by tracing each path back to the first node after the start. The Link State Algorithm will first initialize a table for node distances, previous nodes that are linked to the current node, and a priority queue.

The Link State Algorithm begins by initializing a table for all nodes, previous hops, and a priority queue. It will then start at the chosen node and will only consider that node to begin with. After that, it will expand the graph outward by repeatedly selecting the node that can currently be reached with the lowest total cost. Each time, it will examine only the neighbors of the selected node, to ensure that nodes are processed only after being discovered by an existing connection.

The priority queue is used to determine which node should be analyzed next and is initialized with just the starting node with a cost of zero. It will store pairs consisting of a node and the current best path cost to reach the specified node from the source. As the algorithm continues, adjacent nodes are added to the queue with their total costs. If a better path to a node is discovered later, it will be inserted into the priority queue with the new cost and path. The queue maintains the correct order so that the node with the smallest cost is always processed first. Any outdated entries that represent higher costs are ignored when removed from the queue.

Costs are calculated by adding the weight of an edge to the total cost required to reach the current node. If this newly calculated cost is lower than the previously stored cost for a neighboring node, the table is updated, and the current node is recorded as the previous node. This process ensures that the stored distances always represent the minimum possible cost paths.

During the main loop, the algorithm repeatedly removes the node with the smallest cost from the priority queue and its neighboring edges. Once the queue is empty, the algorithm has determined the shortest path from the starting node to every node and constructed a shortest-path tree. Finally, the program outputs a routing table in the format (node, via, distance) based on these results.

Results:

The following will use a mock of the graph provided by the assignment. It is stored in a txt file called 'ex_graph.txt', using the same format as a list of commands to enter new nodes or edges

```
> python main.py .\ex_graph.txt
Graph loaded successfully:

A: {'D': 5, 'B': 2, 'E': 6}
D: {'A': 5, 'F': 2, 'E': 4}
F: {'D': 2, 'E': 1, 'G': 7}
B: {'A': 2, 'C': 1}
E: {'A': 6, 'D': 4, 'F': 1, 'C': 3}
G: {'F': 7, 'H': 3}
C: {'B': 1, 'E': 3, 'H': 1}
H: {'C': 1, 'G': 3}
Enter nodes in the format 'X Y cost', pressing enter after each. A cost of
'-' will delete the edge
To view your graph, type 'view'
To clear your graph, type 'clear'
To run linkstate, type 'ls X' to generate all paths from node X to all other
nodes
To run an iteration of distance vector, type 'dv X' to print the routing
table for node X, or just 'dv' to run an iteration and print all routing
tables
To exit, type 'done' or 'exit'
>>>
```

From this point, you have the option of several commands:

```

>>> clear
>>> view
>>> A B 1
>>> view
A: {'B': 1}
B: {'A': 1}
>>> B C 10
>>> view
A: {'B': 1}
B: {'A': 1, 'C': 10}
C: {'B': 10}
>>> C B -
>>> view
A: {'B': 1}
B: {'A': 1}
C: {}
>>>

```

Clear: Clears the current graph.

View: Prints the current state of the graph.

X Y cost: Creates new nodes or edits existing nodes in graph.

X Y -: Removes the edge between nodes X and Y

You can also exit the program by running 'exit' or 'done'. You can run any of the above commands at any time, including between distance vector computations or link state computations.

In order to run the Link State algorithm, you can run 'ls X' where X is whichever node you want as the source. It will then print out the distance to every other node from that node, and the one node before this destination node. To demonstrate, I will run it with the initial demonstration graph:

```

>>> view
A: {'D': 5, 'B': 2, 'E': 6}
D: {'A': 5, 'F': 2, 'E': 4}
F: {'D': 2, 'E': 1, 'G': 7}
B: {'A': 2, 'C': 1}
E: {'A': 6, 'D': 4, 'F': 1, 'C': 3}
G: {'F': 7, 'H': 3}
C: {'B': 1, 'E': 3, 'H': 1}
H: {'C': 1, 'G': 3}
>>> ls A
Link-State Routing Table for A
-----
A - 0
B A 2
C B 3
H C 4
D A 5
E A 6
F D 7
G H 7
>>>

```

Here, you can see that it found the shortest distance to each node from A and printed out the node before it in the path to the destination node. We can cross-reference this with the Link State implementation above and see that the final costs and paths are correct.

We can also run 'dv X' in order to run one iteration of the distance vector algorithm, and print the routing table for node X, or we can run just 'dv' to print all routing tables. Below I will run dv four times, to demonstrate how the distance information propagates through nodes.

```
>>> view
A: {'D': 5, 'B': 2, 'E': 6}
D: {'A': 5, 'F': 2, 'E': 4}
F: {'D': 2, 'E': 1, 'G': 7}
B: {'A': 2, 'C': 1}
E: {'A': 6, 'D': 4, 'F': 1, 'C': 3}
G: {'F': 7, 'H': 3}
C: {'B': 1, 'E': 3, 'H': 1}
H: {'C': 1, 'G': 3}
```

```
>>> dv
```

Routing Table for each Node (routing table is below node header)

Routers:	A	B	C	D	E	F	G	H
A	0	2	3	5	6	7	inf	inf
B	2	0	1	7	4	inf	inf	2
C	3	1	0	7	3	4	4	1
D	5	7	7	0	3	2	9	inf
E	6	4	3	3	0	1	8	4
F	7	inf	4	2	1	0	7	10
G	inf	inf	4	9	8	7	0	3
H	inf	2	1	inf	4	10	3	0

After the first run, we see that many nodes are deemed “unreachable” due to infinite distance, as the nodes with these infinite distances do not have any neighbors who neighbor the unreachable nodes. Therefore, the distance to these nodes has not yet reached them, causing infinite distance.

```
>>> dv
```

Routing Table for each Node (routing table is below node header)

Routers:	A	B	C	D	E	F	G	H
A	0	2	3	5	6	7	14	4
B	2	0	1	7	4	5	5	2
C	3	1	0	6	3	4	4	1
D	5	7	6	0	3	2	9	8
E	6	4	3	3	0	1	7	4
F	7	5	4	2	1	0	7	5
G	14	5	4	9	7	7	0	3
H	4	2	1	8	4	5	3	0

After the second run, there are not more unreachable nodes. For instance, G and H become reachable by A. This is because after the last run, the routing table for D updated with the routing table for F, which can reach G. So, in D, the routing table's distance to G read 9 - the distance from G to F to D. Now, when A reads the routing table from D, it has a distance, and adds the distance to D to get 14. A similar process happens for all the rest.

```
>>> dv
```

```
Routing Table for each Node (routing table is below node header)
```

=====								
Routers:	A	B	C	D	E	F	G	H

A	0	2	3	5	6	7	7	4
B	2	0	1	7	4	5	5	2
C	3	1	0	6	3	4	4	1
D	5	7	6	0	3	2	9	7
E	6	4	3	3	0	1	7	4
F	7	5	4	2	1	0	7	5
G	7	5	4	9	7	7	0	3
H	4	2	1	7	4	5	3	0

A few distances changed here, as a longer but lower cost path was able to be found. For instance, A found that G to H to C to B to A was less costly than the last way, and updated its path.

```
>>> dv
```

```
Routing Table for each Node (routing table is below node header)
```

=====								
Routers:	A	B	C	D	E	F	G	H

A	0	2	3	5	6	7	7	4
B	2	0	1	7	4	5	5	2
C	3	1	0	6	3	4	4	1
D	5	7	6	0	3	2	9	7
E	6	4	3	3	0	1	7	4
F	7	5	4	2	1	0	7	5
G	7	5	4	9	7	7	0	3
H	4	2	1	7	4	5	3	0

```
>>>
```

In the previous run, we found all the minimum values, as the routing tables for each node have propagated through to all other nodes. Without changes to the graph, the routing tables will no longer change.

One last thing to display is how the routing tables and distance vector algorithm react when a connection between a node and another node is disrupted. For instance, if we remove the connection from B to A:

```
>>> A B -
>>> view
A: {'D': 5, 'E': 6}
D: {'A': 5, 'F': 2, 'E': 4}
F: {'D': 2, 'E': 1, 'G': 7}
B: {'C': 1}
E: {'A': 6, 'D': 4, 'F': 1, 'C': 3}
G: {'F': 7, 'H': 3}
C: {'B': 1, 'E': 3, 'H': 1}
H: {'C': 1, 'G': 3}
>>> dv
```

Routing Table for each Node (routing table is below node header)

Routers:	A	B	C	D	E	F	G	H
A	0	0	9	5	6	7	13	10
B	10	0	3	7	4	5	5	2
C	9	0	0	6	3	4	4	1
D	5	0	6	0	3	2	9	7
E	6	0	3	3	0	1	7	4
F	7	0	4	2	1	0	7	5
G	13	0	4	9	7	7	0	3
H	10	0	1	7	4	5	3	0

```
>>> dv
```

Routing Table for each Node (routing table is below node header)

Routers:	A	B	C	D	E	F	G	H
A	0	0	9	5	6	7	13	10
B	10	0	3	7	6	5	5	4
C	9	0	0	6	3	4	4	1
D	5	0	6	0	3	2	9	7
E	6	0	3	3	0	1	7	4
F	7	0	4	2	1	0	7	5
G	13	0	4	9	7	7	0	3
H	10	0	1	7	4	5	3	0

```
>>> dv
```

Routing Table for each Node (routing table is below node header)

Routers:	A	B	C	D	E	F	G	H
A	0	0	9	5	6	7	13	10
B	12	0	5	7	6	7	7	4
C	9	0	0	6	3	4	4	1
D	5	0	6	0	3	2	9	7
E	6	0	3	3	0	1	7	4
F	7	0	4	2	1	0	7	5
G	13	0	4	9	7	7	0	3
H	10	0	1	7	4	5	3	0

Here, we can see that on the first iteration, the two nodes that used to be connected to B, A and C, immediately have large changes. These will propagate through to all other routers., and the B section for each table will continuously increase to infinity across all routers.

What went well:

The coding portion of the project went well, as we were able to get together early and get started quickly. Working in Python also made it a bit easier to set up. The Link State Routing was especially able to be implemented without many problems, as we had had some experience implementing Dijkstra's algorithm beforehand. We were also able to parse user input and file input into a graph pretty quickly, and we were easily able to put our work together to get everything working.

What we learned:

We learned a lot about Distance Vector Routing, as it was still pretty unclear how it worked to use when we began the project. In particular, it was confusing how the new routing tables and knowledge propagated through to other routers, and what happened when a router dropped its connection, which we later went over in class. In regards to the Link State Algorithm, we learned how to obtain the last node hop, and its distances as well as implementing Dijkstra's Algorithm.

What was hard:

Wrapping our heads around the Distance Vector Routing algorithm was difficult at first, but it made a lot more sense after some more research and going to the next class. It was also difficult to properly implement it so that it would showcase the proper "counting to infinity" problem if all paths to a node were deleted. The Link State algorithm also initially gave some confusion, but again this was quickly resolved with some more research on how it worked.

What did we not complete from our initial design:

We were able to meet all of our goals from the initial design and standard project description.